

Conformal prediction for multi-class classification in football matches results

Index

Introduction.....	3
Data description	4
Preprocessing.....	5
Feature engineering.....	5
Preprocessing Pipeline	5
Baseline models	5
1. Random Forest	6
2. Logistic Regression	6
3. Multi-Layer Perceptron (MLP).....	6
4. LightGBM	7
5. Gaussian NB	7
6. LDA.....	7
7. QDA.....	7
8. SVM RBF	7
Validation and initial results	8
Feature importance.....	8
Error decomposition.....	9
Test results	10
Conformal prediction	10
Calibration	11
Coverage.....	11
Interpretation of predictive sets	12
Example of interpreting a prediction	12
Discussion.....	13
Comparison between methods.....	13
Impact of conformal prediction.....	13

Limitations	13
Possible improvements	13
Conclusions and future research.....	14
References	15

The main objective of the project is to study the prediction of match outcomes in “La Liga Hypermotion”, which is Spain’s football second division, based on the last 5 seasons (2020-2025). The results could be classified into one of the three following groups: home team win (H), draw (D) and away team win (A). To achieve this, a feature engineering pipeline has been designed, based on historical metrics (rolling windows of points, goals, and team form), also, we have used stadium geolocation data and each club’s squad market value for each season, to make the dataset more complete.

We compared the performance of several models: random forest, logistic regression, Multi-Layer Perceptron (MLP), LightGBM, the generative models Gaussian Naïve Bayes, LDA (Linear Discriminant Analysis) and QDA (Quadratic Discriminant Analysis), a SVM (Support Vector Machine) with RBF kernel and a MLP Neural Net.

To do this, first we did a stratified 5-fold validation. With the stratification, we make sure that we have the same proportion of each class in every fold in the sets of training, validation and test, and thanks to that, we will avoid biasing the evaluation towards majority classes. On the other hand, by using k-folds what we achieve is that, as we rotate the data subset to act as the validation set in each iteration, we estimate the unconditional error by averaging the error from each iteration, thereby reducing the variance of the risk estimate. We chose to use 5 folds because it gives us enough repeats for averaging without blowing up computation time.

We must point out that we only have one parametric model (logistic regression) so we can only compute the values for the AIC and BIC for that model. If we do not compare those values with others, we cannot extract much information or make any conclusions at all about which model is better, so we will have to figure it out by studying the values of the metrics accuracy, recall, F1...

In validation, logistic regression achieved the highest F1_macro of all models (0.397), closely followed by Random Forest (0.367).

On the test set, overall accuracy was 40.5% and F1_macro was 0.397, which makes more obvious the problem’s complexity and the strongest indicator of how the match will conclude is the fact of playing or not at home.

To provide coverage guarantees and quantify the uncertainty of each prediction, multiclass conformal prediction was implemented ($\alpha = 0.1$). After calibrating on the validation set, an empirical coverage of 89.12% and an average predictive set size of 2.48 classes were obtained, reflecting a trade-off between point accuracy and probabilistic reliability.

Here, $\alpha = 0.1$ means that we tolerate a 10% miscoverage rate in long-run predictions; an empirical coverage of 89.12% means that the true outcome falls within the predicted set in 89.12% of cases; and an average set size of 2.4834 classes indicates that each prediction set contains about 2.4834 possible outcomes on average, balancing specificity and uncertainty.

Introduction

La Liga Hypermotion is Spanish’s football second division, and it is known for its uncertainty. Teams which are at the top of the table usually loose against teams that are trying to avoid relegation, in fact, parity and intense rivalry are the main characteristics of the league.

The aim of the project is to develop and evaluate machine learning pipelines which can give a match, return a prediction for the match outcome, with robust performance metrics and uncertainty guarantees.

The key questions that will be addressed in this project are the following:

1. Which model family offers the best trade-off between accuracy and generalization on this dataset?
2. How does incorporating stadium geolocation and squad market value impact predictive performance?
3. What level of coverage and average predictive set size is achieved when applying conformal prediction with $\alpha = 0.1$?

Data description

We have used three different datasets:

1. “liga_hypermotion_2020-2025”: this dataset includes information about all the matches that have been held between 2020 and 2025 (each match has its own identifier).
2. “team_locations”: it includes the coordinates associated with each team that has been in the league during the 5 seasons studied.
3. “team_values”: it has the values of the squads of all teams between the years 2020 and 2025.

The features of the datasets that were used are the following:

- “result”: it takes one of the following values: H, D or A. H stands for the victory of the local team, D for a draw, and A for the victory of the away team.
- “home_goals” and “away_goals”: are the number of goals that have been noted by each of the teams in the match.
- “*_form_points_last5”, “*_form_winrate_last5” and “*_conceded_last5”: they describe the form of the local/ away team. They tell how many points the team has scored/ goals the team has noted / the goals the team has conceded in the last 5 matches.
- “home_latitude”, “home_longitude”, “away_latitude” and “away_longitude”: they are the coordinates of the locations of each team, so thanks to that information, we can compute the magnitude of the journeys the team has made.
- “home_market_millions” and “away_market_millions”: they tell the value of the squad in terms of millions of euros.

Also, we have other features that were important for the structure of the dataset, but did not were relevant in terms of training our models. The main features of this type are:

- match_id: specific identifier for each match that has taken place in the last five seasons.
- “date”: contains the date of the match (thanks to it we could order the matches chronologically).
- “season”: the season in which the match took place.
- “result”: it contains the real result of the match (H, D or A).

Preprocessing

As not all the features were relevant and the raw data was not the optimal, a preprocessing of the data was made. The steps followed are listed below:

1. Duplicate removal: Rows duplicated across merged tables were discarded.
2. Date conversion: The date column was parsed as datetime to ensure chronological ordering.
3. Handling missing values: Stadium coordinates and market values were initially set to NaN during merging and subsequently filled with zeros.
4. Leakage filtering: Variables directly tied to the outcome (home_points, away_points, bet_odds_*, home_goals, away_goals) were removed.
5. Label normalization: The result field was encoded to result_enc using LabelEncoder for model training.

Feature engineering

As we wanted to capture temporal performance trends, spatial dynamics, and squad strength, we constructed a comprehensive suite of derived features:

- Rolling windows of points and goals (W = 5): for both home and away teams, the mean points and goals scored/conceded over the previous five matches were computed to reflect recent form.
- Form and win-rate calculation: from the rolling windows, we extracted (a) average points (*_form_points_last5) and (b) win-rate (*_form_winrate_last5) over the last five games.
- Stadium distance: using home_latitude, home_longitude, away_latitude, and away_longitude, we calculate the distance between stadiums so that we can have an idea of the fatigue each team will carry into the recovery week.
- Market value as an indicator of the squad quality: the aggregated market value in millions (*_market_millions) serves to approximate each team's financial and talent resources.
- Leakage filtering: any features that directly encode the match outcome (e.g., home_points, away_points, betting odds, raw goal counts) were excluded to prevent target leakage.

Preprocessing Pipeline

To ensure robust model training and fair evaluation, the following preprocessing steps were applied:

1. Median imputation: missing values in numeric features (coordinates, market value) were replaced with the median computed on the training set.
2. Standard scaling: all features were standardized to zero mean and unit variance using StandardScaler fit only on the training fold.
3. Data partitioning: The dataset was split chronologically into training+validation (85%) and hold-out test (15%). The training+validation set was further stratified into training (70%) and validation (30%), resulting in an overall 15% test and 15% validation split.

Baseline models

In this section we will present the models evaluated as our baselines.

Each model has been trained on the pre-processed features and optimized with hyperparameters tuned via cross-validation where applicable.

Validation results are reported using 4 metrics: accuracy, recall, F1 and AUC-ROC.

- Accuracy: the proportion of correct predictions out of all cases. It measures how often the model gets it right, without distinguishing between classes.
- Recall: of all the samples that belong to the positive class, what fraction the model successfully captures. It is relevant if I want to make sure that I do not miss any important positive.
- F1: the harmonic mean of Precision and Recall. It gives you a single number that balances getting hits and covering positives; if one drops, F1 falls hard too.
- AUC_ROC: the area under the Receiver Operating Characteristic curve. The ROC plots the true positive rate (TPR) against the false positive rate (FPR) as you vary the decision threshold. A value of 0.5 indicates random guessing, while 1.0 means the model is perfect.

The models that have been used are the ones shown below:

1. Random Forest

Random forest builds an ensemble of decision trees, each of the mis trained on a bootstrapped sample of the data and considering a random subset of features at each split. Predictions are made by aggregating across all trees (majority voting), and this reduces variance and mitigates overfitting if we compare the performance to the one that we would obtain with a single tree. The model is robust to overfitting and captures non-linear feature interactions, but sensitivity to hyperparameters is moderate.

The setup that was used was the following: 300–600 trees, maximum depth 6–30, random search over splits and leaf sizes and class weights balanced.

2. Logistic Regression

Logistic regression models the log-odds of each class as a linear combination of input features and applies the logistic (softmax) function to obtain class probabilities. Coefficients are learned by maximizing the likelihood of the observed labels, providing interpretable feature weights and effective regularization through L2 penalty.

It is a simpler linear model that generalized best on this dataset; likely benefits from the high dimensionality and standardized inputs.

The setup that was used was the following: multinomial loss, L2 regularization via default C, `class_weight='balanced'`.

3. Multi-Layer Perceptron (MLP)

An MLP is a feed-forward neural network composed of an input layer, one or more hidden layers, and an output layer. Each layer applies a linear transformation to its inputs followed by a non-linear activation (e.g., ReLU). During training, the network learns weight matrices and bias vectors by minimizing a loss function (cross-entropy for classification) using gradient-based optimization (backpropagation with Adam). This allows the model to capture complex non-linear relationships among features.

We obtained a lower performance compared to the previous models probably due to the limited data size, fixed architecture, and minimal hyperparameter tuning.

The setup that was used was: two hidden layers with 128 and 64 neurons respectively, ReLU activation, Adam optimizer and trained for 300 epochs with default L2 regularization.

4. LightGBM

LightGBM is a gradient boosting framework that builds decision trees sequentially. Each new tree is trained to correct the errors of the previous ensemble by minimizing a loss function (multi-class logloss in this study). LightGBM uses histogram-based binning for fast computation, leaf-wise tree growth for higher accuracy, and supports efficient handling of large datasets with feature parallelism and bagging.

The model performs well with non-linear interactions and large feature sets.

The setup that was used was: 800 boosting rounds, learning rate 0.05, 63 leaves, subsample=0.8, colsample_bytree=0.8 and class weights balanced.

5. Gaussian NB

It is a generative probabilistic classifier that assumes that each feature, given the class, follows a Gaussian distribution. It treats the features as conditionally independent and uses the Baye's theorem to compute the probability of each class given an observation vector.

6. LDA

It is a generative model which assumes that the observations of each class are generated by a multivariate Gaussian with the same covariance matrix. From this, it derives an optimal linear decision boundary by maximizing the between-class vs. within-class variance ratio.

It works really well when the classes are linearly well separated.

7. QDA

It's a version of LDA that relaxes the shared-covariance assumption: each class has its own covariance matrix. This leads to quadratic decision boundaries, which are more flexible for capturing nonlinear separations.

8. SVM RBF

It is a discriminative classifier that finds the maximum-margin hyperplane in a transformed feature space. The RBF kernel (radial basis function) maps data without computing explicit coordinates, letting you discover highly nonlinear boundaries. It's usually robust across many problems, but you need to tune the RBF width parameter (γ) and the penalty parameter (C).

In this project $\gamma = 1 / (\text{number of features} \cdot \text{Var}(X))$ and $C=1.0$ (we used the default value). With these values, the model strikes a standard balance between complexity and regularization.

In the following table we can see the values of the four metrics with each model that has been explained before.

Metric / model	Logistic regression	Random Forest	MLP	LightGBM	Gaussian NB	LDA	QDA	SVM RBF
Accuracy	0.405	0.402	0.372	0.381	0.390	0.423	0.350	0.441
Recall	0.404	0.369	0.345	0.348	0.365	0.360	0.330	0.356
F1	0.397	0.367	0.343	0.347	0.355	0.340	0.330	0.303

AUC-ROC	0.547	0.532	0.515	0.534	0.538	0.540	0.516	0.512
----------------	-------	-------	-------	-------	-------	-------	-------	-------

If we compare the performance of the eight models, we see that Logistic Regression wins the consistency game: it balances precision and coverage, with an F1 of nearly 0.40 and an AUC-ROC of 0.547. That means, even if it's not the one with the best value for accuracy, it inspires the most confidence by spreading its hits evenly across all three classes and discriminating more reliably.

Then, the SVM with RBF kernel shows up as the most “accurate” in terms of pure accuracy (0.441), but its F1 tanks down to 0.303 and its AUC-ROC drops to 0.512. In practice, this means that, although it gets more right overall, it focuses so heavily on the majority class that it loses its edge on the others.

On the flip side, the generative models (Gaussian NB and LDA) prove surprisingly competitive in AUC (0.538 and 0.540, respectively).

The boosting and bagging techniques (Random Forest and LightGBM) sit on solid ground: RF achieves an F1 of 0.367 and an AUC of 0.532, while LightGBM hits an F1 of 0.347 and AUC of 0.534. They're not groundbreaking, but they offer rock-solid performance for automated pipelines.

Finally, the MLP and QDA stumble a bit, with F1s under 0.35 and AUCs around 0.515–0.516, which indicates that without more data or fine-tuning they tend to fall short on this three-class challenge.

To conclude, the logistic regression was the strongest model and the one with the best performance on the validation set, and could balance between simplicity and generalization, which led to its selection for the conformal prediction phase.

Validation and initial results

In this section we will summarize the validation performance of all the models that have been presented, providing initial insights before final testing.

The logistic regression was the model that achieved the highest score for the F1 metric, closely followed by the random forest. LightGBM and MLP had a really poor performance while stacking did not improve over the best individual models.

As we obtained these results, the model selected for the final training and the conformal prediction was the logistic regression.

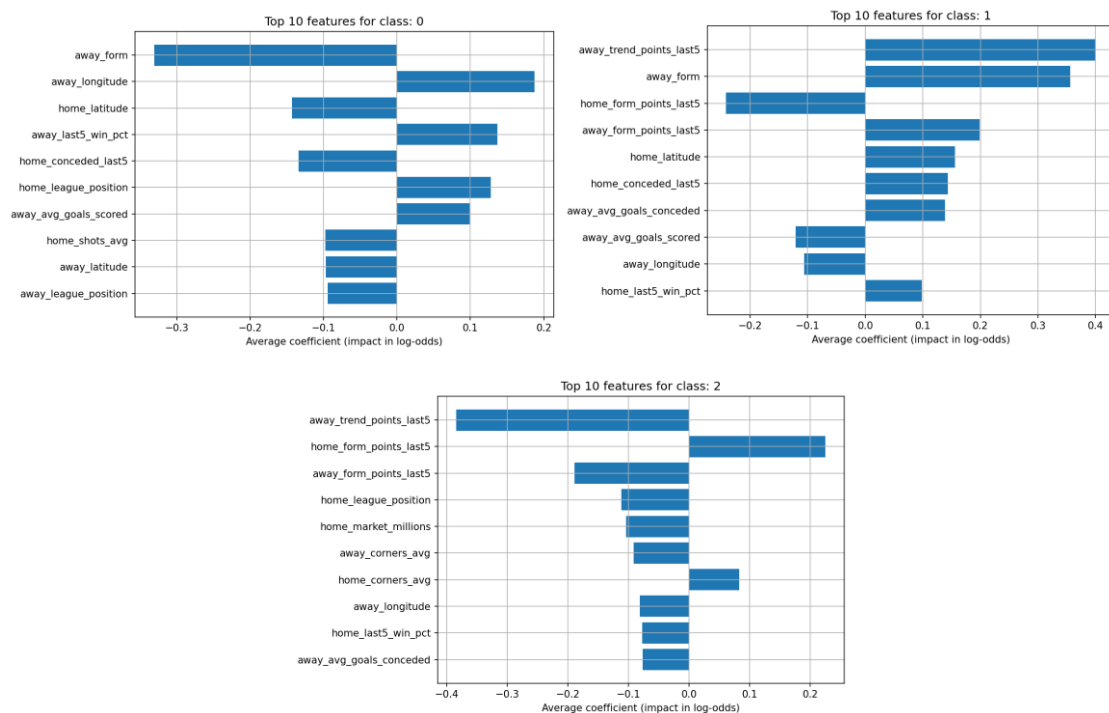
Feature importance

To make our model more interpretable, we will extract the learned features coefficients, by extracting the average coefficients from the 5 models calibrated during cross-validation.

Each coefficient reflects the influence of a feature on the log-odds of each outcome class. In this model the top features for home wins (class 0) included the away's team recent form (away_form), and their longitude and win rate in the last 5 matches. This indicates that stronger away teams reduce the likelihood of a home win. The stadium geolocation and the home team shooting statistics also had a relevant role.

Then, for the draws (class 1) the features that have more importance were away trend points in the last 5 matches, overall away form and balanced metrics from both teams such as form points and conceded goals, and this reveals that inconsistent teams tend to end in draws. Also, we must highlight that `home_form_points_last5` had a negative impact, so, this says that home consistent teams are less likely to draw.

For away wins (class 2), on the other hand, its most relevant features were the away trend points in the last 5 matches, followed by home form (negatively correlated), and this shows that declining home team performance combined with strong away momentum leads to a higher likelihood of away victories. Also some economic and tactical features such as the market value and corners also seemed relevant.



Error decomposition

When we break down the error, we see that the bulk of the failure doesn't come from wild instability between trainings but from a high, consistent bias. With a Bias² around 1.06 versus a variance of just 0.15, our model is failing in almost the same way every time: it cannot capture the real complexity of the data.

This insight tells us we don't need to clamp down on overfitting. Maybe features that tell extra stories (interaction terms, live game data, weather) would give it the expressiveness it's lacking today. In other words, the move is to enrich and sophisticate, not add more restrictions. That way we can drive that bias down to deliver predictions that really hit the mark.

Test results

After selecting the logistic regression as the final model, it had to be retrained on a set that was the combination of the validation and training sets, which conformed the 85% of the total data. And then, it was evaluated on the remaining 15% of the data, which was the test set.

The key performance metrics were:

- Accuracy: 40.18%. Only about two out of five predictions are correct, reflecting the challenge of three-class classification.
- F1_macro: 0.394. The macro-averaged F1 score balances per-class performance, indicating moderate success across home wins, draws, and away wins.

Class/ Metric	Precision	Recall	F1-score	Support
H	0.45	0.91	0.60	147
D	0.27	0.03	0.05	101
A	0.43	0.11	0.17	83
Macro avg	0.38	0.35	0.28	331

In the table that has just been shown we can see the values of different metrics corresponding to each of the classes that can be predicted and the macro average.

The main conclusions we can extract from this are that the model has a better performance on home wins, as the corresponding F1-score has a value of 0.60, which is the biggest of all of them, also, it has a poorest performance with draws, and we can see that it's F1-score is equal to 0.05.

Also, we can observe the corresponding confusion matrix to understand what is happening with the classification:

	A	D	H
A	9	4	70
D	21	3	77
H	6	7	134

The columns of the confusion matrix correspond to the predicted classes, and the rows to the actual value of them.

Here we can see that the biggest part of home wins is classified correctly, while a huge part of away wins and draws are misclassified as home wins, which shows once again the problem that we have with the bias.

The confusion matrix makes more obvious that our model tends to classify the result of the matches as home wins.

Conformal prediction

First of all, what is “conformal prediction”? Why is it relevant?

We can see conformal prediction as a “shield of trust” that can be added to our machine learning model, in order to gain confidence in our prediction. Instead of giving us a concrete number it will return us a certain range, guaranteed with a preset confidence level. For example, if we set the

confidence interval in 90%, we know that if we repeat the experiment 100 times, in 90 of those repetitions, the results will be inside the given range.

Once we have selected “logistic regression” as the model which give us better results in this problem, we do the following:

1. We train a new logistic regression model with train data.
2. Then, we separate a small amount of data from the data that has not been used to train the model, and we use it to see how strange are the predictions that the model makes.
3. We order the scores and pick the percentile that matches the confidence level we want (for example, if we are looking for a 90% confidence level, we will choose the 90% percentile).
4. Finally, in future predictions, we will build a range around the point estimate, using that threshold score.

“Conformal prediction” works with every kind of model (but in this case we will use it with the logistic regression), and thanks to it we can gain robustness. Also, it ensures that our intervals satisfy the confidence Interval no matter what and it is really transparent, as in every moment we will know how wide the interval is depending on the confidence level that we set.

Now, we will have to see the calibration step and talk about the coverage.

Calibration

As we want to have a 90% confidence level, we set alpha equal to 0.1 ($\alpha = 0.1$).

In the calibration step we set a threshold value (we will call it $\hat{q}$), and this value will separate “likely” from “unlikely” predictions. The steps that have been followed are:

1. We have divided the data into three groups: training, validation and test. After training our model, we use the validation set for calibrating the model.
2. For each example in the calibration set, we compute the model’s probability estimation for each class.
3. We calculate a score for each class: $s_i = 1 - \hat{p}(y_i|x_i)$ where $\hat{p}(y_i|x_i)$ is the probability assigned to the correct class. If we see lower scores, that means that we have higher confidence.
4. Finally, we sort all nonconformity scores and choose the value at the 90th percentile (since $\alpha = 0.1$). This threshold ensures that, over the long run, no more than 10% of true labels will be excluded from their predictive sets.

With that $\hat{q}$ we build predictive sets by including all classes whose nonconformity score is lower or equal to $\hat{q}$.

By setting alpha equal to 0.1, we strike a balance between narrower prediction sets and a targeted 90% coverage.

Coverage

What is the empirical coverage? It measures the proportion of test instances for which the true class label is included in the predictive set $C(x)$.

With alpha equal to 0.1 we obtained the following results:

- An empirical coverage equal to 89.12%, so, that means that in a 89.12% of the cases, our predictive sets contained the correct outcome.
- 2.4834 classes per instance, and this means that most predictions are doubletons (size=2), so the model is confident to predict two (or more) classes.

As we obtained a really good result for alpha equal to 0.1, we did not have the necessity to sweep alpha over other values. Choosing alpha equal to one gives us a reliable coverage while still providing somewhat sharp (usually two-class) predictive sets.

Interpretation of predictive sets

Conformal Prediction outputs a set of possible labels for each instance rather than a single point prediction. The set can have three possible sizes:

- Singleton sets (size =1): in this case the predictive set contains only one class (for example, {H}). This means that the predicted class has much higher probability than the other classes. In this case, our model does not feel certain about one outcome.
- Doubleton sets (size = 2): the set contains two classes (for example {H, D}). When we see this, it means that two classes have exceeded the confidence threshold, and the model could not choose between one of them.
- Tripleton sets (size = 3): the set contains three classes ({H, D, A}) and this means that all three classes are equally likely. It is the rarest case and indicates greater equality.

By examining the distribution of set sizes across the test set, stakeholders can:

1. Adjust risk tolerance: choose decisions only when singletons occur or accept doubletons when broader coverage is needed.
2. Allocate attention: focus manual review on matches with doubleton/triplet sets, where model uncertainty is highest.
3. Tune α : if too many doubleton/triplet sets appear, increase α to widen sets and boost coverage, or decrease α to force more singletons.

Example of interpreting a prediction

If we want to predict the outcome of the following match: Tenerife – Oviedo (2025-05-25), the model returns this:

Result	Home win	Draw	Away win
Probability	0.25	0.312	0.438

Our model sees Oviedo as slight favorite (43.8 %), with draw at 31.2 % and Tenerife at 25 %. But because none of those probabilities clears our confidence threshold, conformal prediction returns all three outcomes, signaling maximal uncertainty.

In other words, the model's telling us, "Oviedo leads, but I can't rule anything out with 90 % confidence." Practically, this means you shouldn't auto-commit to any result here: it would be recommendable to grab extra info (lineups, weather) or defer to expert judgment.

Discussion

In this section we will compare all the evaluated methods, we will analyse the impact of the conformal prediction and finally we will show the limitations of the study and possible improvements that can be made in the future.

Comparison between methods

Logistic Regression showed the best balance between simplicity and generalization, hitting a solid macro-F1 of 0.361 in validation and 0.397 on the test set. Its edge comes from being super interpretable and needing less data to train stable models.

Random Forest delivered a close performance (macro-F1 = 0.367), better capturing non-linear interactions but carrying a higher overfitting risk without deep tuning.

LightGBM and MLP posted moderate scores (0.347 and 0.343, respectively), suggesting the dataset's signal isn't strong enough to justify more complex models without boosting data quality and quantity.

LDA and Gaussian NB reached an AUC near to 0.54, which makes them quick allies for estimating uncertainty.

Impact of conformal prediction

The application of conformal prediction added a quantifiable layer of uncertainty, and that letted us building predictive sets with formal coverage guarantees.

With $\alpha = 0.10$ we achieved an empirical coverage of 89.12% and an average set size of 2.48 classes. With this value for alpha we have already obtained a great coverage, so we did not have to explore more values.

This approach gives us explicit control over the trade-off between point accuracy and probabilistic reliability, and this is perfect for the case where the prediction of matches outcomes is more used: sports betting. In sports betting we must minimize the error risk, since there's real money on the line, so the use of conformal prediction is helpful.

Limitations

We can observe the following limitations:

1. Partial coverage: the coverages obtained are far below the nominal level ($1 - \alpha$), likely due to miscalibrated probabilities and overlapping class distributions.
2. Data quality and quantity: the league dataset has limited features as we have not included factors as the lineups of each match or the weather of the day.
3. Residual leaks: even after cleaning, implicit variables correlated with the outcome can still bias the training.
4. Scalability: calibration and stacking methods demand significant computational resources, limiting rapid experimentation with α and new models.

Possible improvements

The improvements that are shown bellow can be made to obtain a better predictor of the matches:

1. Additional calibration: apply probability calibration techniques (platt scaling, isotonic) before conformal prediction to better align the probability distributions.
2. Feature enrichment: incorporate lineup data, individual player performance, advanced per-minute stats, and weather conditions to boost the predictive signal.
3. Bayesian models: explore probabilistic models that directly estimate output distributions and integrate naturally with conformal prediction.
4. α optimization: design a dynamic α -selection strategy based on each prediction's or match segment's risk profile.

Conclusions and future research

We have seen that a simple model as the logistic regression can predict matches outcomes with a moderate certainty, which is around the 40%, also, it is obvious that playing home is a great advantage.

Then, when we apply conformal prediction, we add a confidence level for each prediction: as we increased the confidence level, we saw that our predictions cover the correct outcome more often, but the set of possible outcomes gets bigger. In short, this approach lets you make safer decisions by showing not only which outcome is most likely, but also how many options you need to consider staying within a given confidence level.

Finally, if we think about possible future research paying attention to the findings that have been made, several directions can be explored:

- Player-level and tactical features: incorporate line-up data, individual player metrics, and in-match events (substitutions, cards) to capture deeper game dynamics.
- External data integration: leverage social media sentiment, betting market movements, and weather forecasts to enrich feature space and potentially improve both accuracy and coverage.
- Probabilistic models: explore Bayesian approaches (e.g., Gaussian Processes, Bayesian Neural Networks) that can naturally output uncertainty estimates and combine them with conformal methods.

References

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>
- Shafer, G., & Vovk, V. (2008). A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(Mar), 371–421.
<https://www.jmlr.org/papers/volume9/shafer08a/shafer08a.pdf>
- Constantinou, A. C., Fenton, N. E., & Neil, M. (2012). pi-football: A Bayesian network model for forecasting Association Football match outcomes. *Knowledge-Based Systems*, 36, 322–339.
<https://doi.org/10.1016/j.knosys.2012.07.008>
- Brutti, P. (2024). *Statistical Learning – Class notes* (Course material, Academic Year 2024/2025). Sapienza Università di Roma.
<https://elearning.uniroma1.it/course/view.php?id=4944>